

南京师范大学

机器视觉与图像处理

(2025~2026 学年第 1 学期)



题	目：	基于数字图像处理与机器视觉的水下大坝裂缝识别
学	院：	电气与自动化工程学院
专	业：	自动化
班	级：	2309 班
姓	名：	赵学文
学	号：	21230933

南京师范大学教务处 制

目录

摘 要	3
一、 引言	4
二、 传统图像处理方法.....	6
2.1 空间域平滑滤波	6
2.1.1 算法原理	6
2.1.2 实现与分析	6
2.2 灰度阈值法分割	8
2.2.1 全局 Otsu 阈值法原理	8
2.2.2 全局分割实验与不足	8
2.3 自适应阈值分割（初步提取裂缝）	9
2.3.1 局部自适应算法原理	9
2.3.2 局部分割实验与分析	9
2.4 数学形态学处理	10
2.4.1 形态学基本算子定义	10
2.4.2 级联去噪与连通策略	10
2.5 轮廓提取与特征量化	11
2.5.1 基于 Five A+ 的增强预处理	12
2.5.2 拓扑轮廓扫描与几何筛选	12
2.5.3 基于迭代内切圆的宽度测量	13
2.5.4 量化统计结果	14
三、 讨论：参数选择、局限与改进方向.....	16
3.1 参数设置的经验性与敏感性说明	16
3.2 改进方向	17
四、 结论	18
五、 参考文献	19
附录：代码仓库与运行说明.....	20

摘要

水下大坝表面裂缝是结构安全巡检中的重要风险信号，但水下成像往往存在对比度不足、颜色偏移与亮度不均，使得裂缝与背景纹理在灰度空间中可分性较差。围绕《数字图像处理与机器视觉》课程知识点，本文以一幅代表性水下图像为例，构建并实现“平滑去噪—阈值分割—数学形态学去伪与连通—轮廓提取与统计”的可解释裂缝候选区域提取流程，逐步展示各处理环节对结果的影响。

在传统流程中，先使用高斯滤波抑制颗粒噪声，再分别采用全局阈值（Otsu）与局部自适应阈值完成初步分割，并利用开/闭运算去除孤立噪声与增强裂缝连通性，最后通过轮廓面积阈值与主裂缝邻域规则筛选裂缝候选并叠加显示。考虑到原始水下图像在阈值分割阶段易出现“碎点多、断裂多”的问题，本文在工程实战部分引入 *FiveA+* 水下图像增强作为前处理，在保持后续传统流程与参数一致的前提下，对比增强前后的统计输出，验证增强能提高裂缝可分性并改善最终轮廓连贯性。

关键词：数字图像处理；机器视觉；阈值分割；数学形态学；水下图像增强

一、引言

1.1 数字图像处理的简介

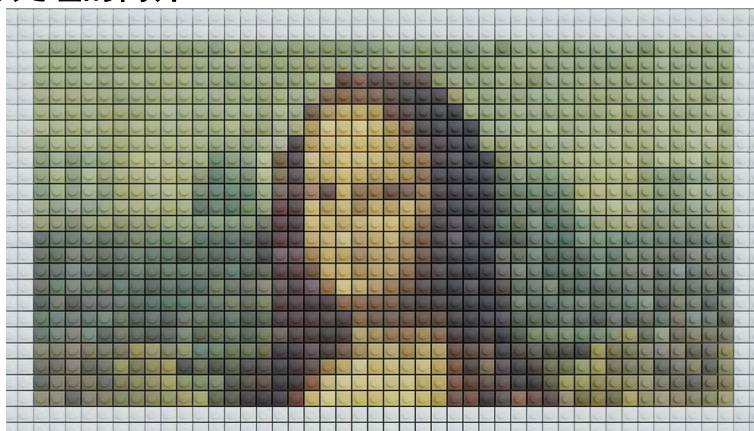


图 1 图像像素化

数字图像处理研究如何将图像表示为计算机可处理的数字信号，并通过算法实现去噪、增强、分割与特征提取等任务。对一幅数字图像而言，可将其理解为由像素构成的二维网格（灰度图对应亮度矩阵，彩色图可视为多通道矩阵），因此许多处理操作都可以归结为对像素及其邻域进行运算。典型方法包括空间域平滑滤波以抑制噪声、阈值分割以把目标从背景中分离、以及数学形态学对二值结构进行去噪与连通修复等。通过这些步骤，可以提升图像可读性，或提高目标与背景的可分离性，为后续的机器视觉分析提供更稳定的输入。

本报告以水下大坝表面裂缝识别为实验场景，按照课程中“预处理—分割—形态学—描绘与统计”的主线完成一套可解释、可复现的传统处理流程，并通过过程图展示每一步对结果的影响。

1.2 主要工作与结构

本报告围绕课程知识点构建一条可解释的裂缝候选区域提取流程，具体工作如下：

1. **流程构建与过程可视化：** 以一幅代表性水下大坝表面图像为例，搭建“平滑去噪—阈值分割—形态学去伪与连通—轮廓提取与统计”的处理链路，并在每个关键环节保存中间结果图，用于对比说明算法对输出的影响。

2. **空间域平滑滤波（去噪）**： 使用高斯滤波抑制颗粒噪声与细碎纹理，降低后续阈值分割的噪声敏感性；在正文中给出滤波前后对比图，并解释核尺寸与平滑强度的选取依据。

3. **阈值分割对比（全局 vs 局部）**： 分别实现 Otsu 全局阈值与局部自适应阈值分割，展示在亮度不均条件下全局阈值的局限性，以及局部阈值在裂缝初步提取上的优势与副作用（碎点增多）。

4. **数学形态学处理（去伪与连通）**： 采用开/闭运算对二值结果进行去噪与连通修复，缓解“碎点多、断裂多”的典型问题；并结合结构元素形状与尺寸，解释其对“去噪力度”和“误连风险”的影响。

5. **轮廓提取与统计指标输出**： 在形态学优化基础上进行轮廓/连通域提取，基于面积阈值等规则筛选裂缝候选区域，将结果叠加回图像进行展示，并输出裂缝数量、面积等统计指标作为定量描述。

6. **工程辅助验证（扩展）**： 针对原始水下图像在阈值分割阶段易出现不稳定的问题，工程实战部分引入水下图像增强作为前处理，以“黑盒调用”的方式使用预训练模型，不展开网络训练细节；在保持传统流程与参数一致的前提下，对比增强前后的统计输出与轮廓连贯性，验证增强对可分性的提升作用。

二、传统图像处理方法

本章以图 2 所示的水下大坝表面图像为输入，依次完成“平滑去噪—阈值分割—形态学去伪与连通—轮廓提取与统计”的裂缝候选区域提取流程。考虑到水下成像常见的纹理颗粒干扰与光照不均会削弱裂缝与背景的可分性，若直接分割往往导致前景碎点增多、裂缝主干断裂。为此，本章将保存并对比展示各处理阶段的中间结果图，用以说明每一步对噪声抑制、连通性修复与最终检测效果的贡献。

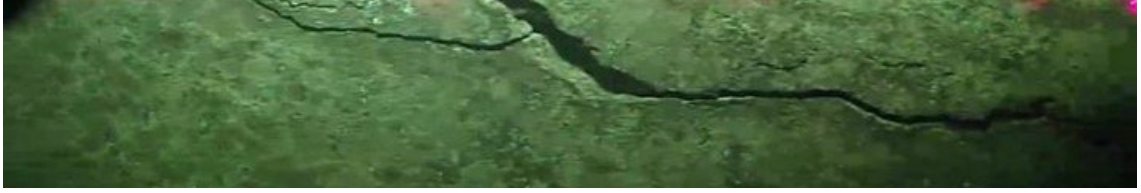


图 2 原始水下大坝表面图像

2.1 空间域平滑滤波

2.1.1 算法原理

由于水下环境中的悬浮颗粒散射，原始采集图像 $f(x, y)$ 往往伴随高频随机噪声。为了在保留裂缝宏观结构的同时抑制噪声，本文采用高斯滤波器（Gaussian Filter）进行空间域平滑。高斯滤波是一种线性平滑滤波，其本质是将输入图像与二维高斯核函数进行卷积运算。

二维高斯核函数 $G(x, y)$ 定义为：

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中， σ 为标准差，控制平滑程度； (x, y) 为相对于核中心的坐标。平滑后的图像 $g(x, y)$ 可表示为：

$$g(x, y) = f(x, y) * G(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b G(s, t) f(x-s, y-t)$$

2.1.2 实现与分析

在实验中，本文选取 (5×5) 的高斯核进行处理（核心代码如代码 1 所示）。相比于均值滤波，高斯滤波根据像素距离中心的远近赋予不同的权重，中心点权重最高，

边缘递减。这种加权机制使得该方法在滤除高频颗粒噪声（Noise Reduction）的同时，能较好地保留裂缝边缘的低频信息。

```
def main() -> None:
    input_path = resolve_input_path()
    image = cv2.imread(str(input_path))
    if image is None:
        raise SystemExit(f"Failed to read image: {input_path}")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    output_path = input_path.parent / "cracks_2_1_smooth.jpg"
    if not cv2.imwrite(str(output_path), blurred):
        raise SystemExit(f"Failed to write image: {output_path}")

    print(f"Saved: {output_path}")
```

代码 1 基于 OpenCV 的高斯平滑滤波算法实现

从下方（图 3）的处理结果可以看出，高斯滤波有效平滑了背景中的细微颗粒感，避免了后续二值化过程中因高频噪点导致的“伪边缘”现象。

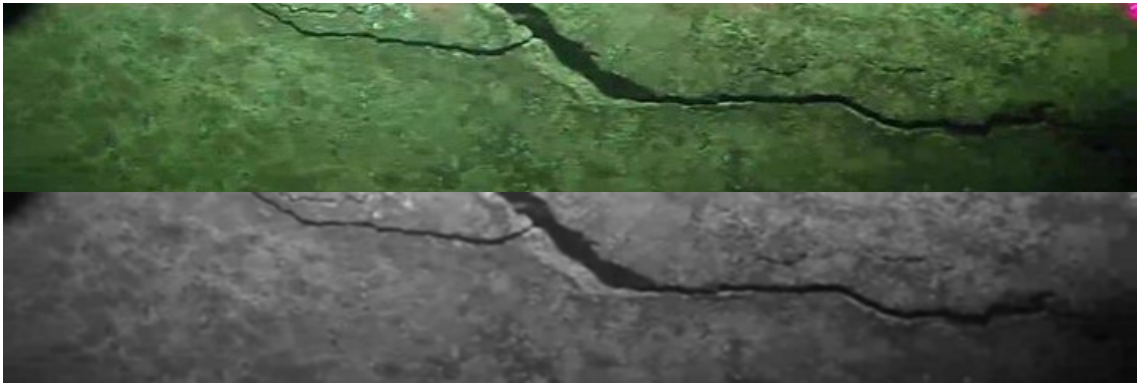


图 3 高斯平滑滤波前后的空间域降噪效果对比

2.2 灰度阈值法分割

2.2.1 全局 Otsu 阈值法原理

图像分割旨在将感兴趣的目标区域（ROI）从背景中分离。Otsu 算法（大津法）是一种基于全局直方图统计的自适应阈值确定方法，其核心思想是寻找最佳阈值 t^* ，使得背景类 C_0 和前景类 C_1 之间的类间方差 $\sigma_B^2(t)$ 最大化：

$$t^* = \arg \max_{0 \leq t < L-1} \sigma_B^2(t) = \arg \max_{0 \leq t < L-1} [\omega_0(t)\omega_1(t)(\mu_0(t) - \mu_1(t))^2]$$

其中， ω 为类概率， μ 为类均值。

2.2.2 全局分割实验与不足

本文首先尝试了全局 Otsu 方法对灰度图进行二值化处理（代码 2）。

```
def main() -> None:
    input_path = resolve_input_path()
    image = cv2.imread(str(input_path))
    if image is None:
        raise SystemExit(f"Failed to read image: {input_path}")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    output_path = input_path.parent / "cracks_2_2_threshold.jpg"
    if not cv2.imwrite(str(output_path), binary):
        raise SystemExit(f"Failed to write image: {output_path}")

    print(f"Saved: {output_path}")
```

代码 2 灰度转换与 Otsu 全局阈值计算

实验结果（如图 4 所示）表明，由于水下光照衰减导致图像整体亮度分布极不均匀，全局单一阈值 t^* 难以兼顾明亮区域与阴暗区域。直接应用 Otsu 方法导致了暗处的背景纹理被误判为裂缝，说明全局方法不适用于非均匀光照场景。

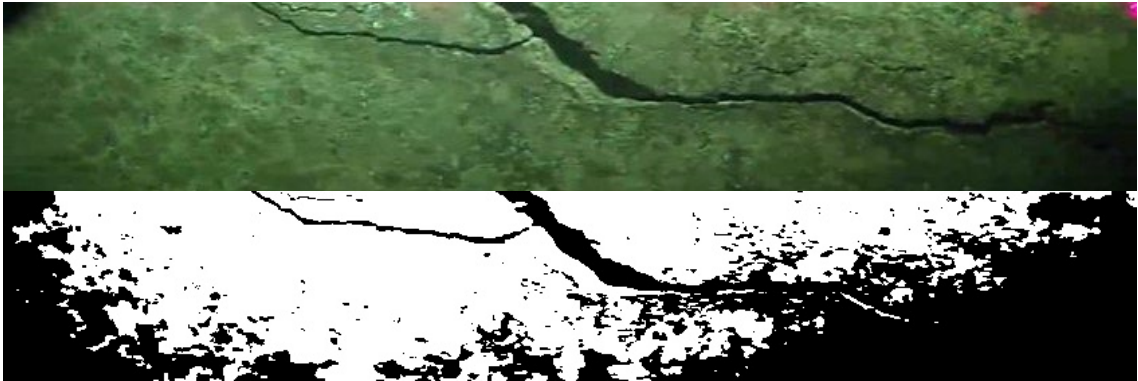


图 4 基于 Otsu 全局阈值的初步分割结果（受光照不均影响）

2.3 自适应阈值分割（初步提取裂缝）

2.3.1 局部自适应算法原理

鉴于全局阈值的局限性，本文进一步采用了局部自适应阈值算法。该方法不再对全图使用固定阈值，而是针对图像中的每个像素点 (x, y) ，根据其邻域 S_{xy} 内的统计特性动态计算阈值 $T(x, y)$ 。本文采用高斯加权平均法计算局部阈值：

$$T(x, y) = \left(\sum_{(i, j) \in S_{xy}} w(i, j) \cdot I(i, j) \right) - C$$

其中， S_{xy} 为以 (x, y) 为中心的局部窗口（实验取 35×35 ）， C 为常数偏移量（Constant Offset）。二值化判决逻辑为：

$$D(x, y) = \begin{cases} 255, & \text{if } I(x, y) < T(x, y) \\ 0, & \text{otherwise} \end{cases}$$

（注：此处采用 THRESH_BINARY_INV，即暗裂缝区域被置为高亮前景）。

2.3.2 局部分割实验与分析

通过缩小判别域，自适应阈值能够有效抵抗低频的光照变化干扰（实现见代码 3）。

```
def main() -> None:
    input_path = resolve_input_path()
    image = cv2.imread(str(input_path))
    if image is None:
        raise SystemExit(f"Failed to read image: {input_path}")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    adaptive = cv2.adaptiveThreshold(
        gray,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        35,
        5,
    )

    output_path = input_path.parent / "cracks_2_3_adaptive.jpg"
    if not cv2.imwrite(str(output_path), adaptive):
        raise SystemExit(f"Failed to write image: {output_path}")

    print(f"Saved: {output_path}")
```

代码 3 局部自适应阈值二值化处理逻辑

实验结果（图 5）显示，该方法成功提取了不同亮度背景下的裂缝主干，解决了 Otsu 方法在暗部失效的问题。但与此同时，该方法也引入了大量由背景纹理细节构成的高频碎点噪声，这需要通过后续形态学操作予以剔除。

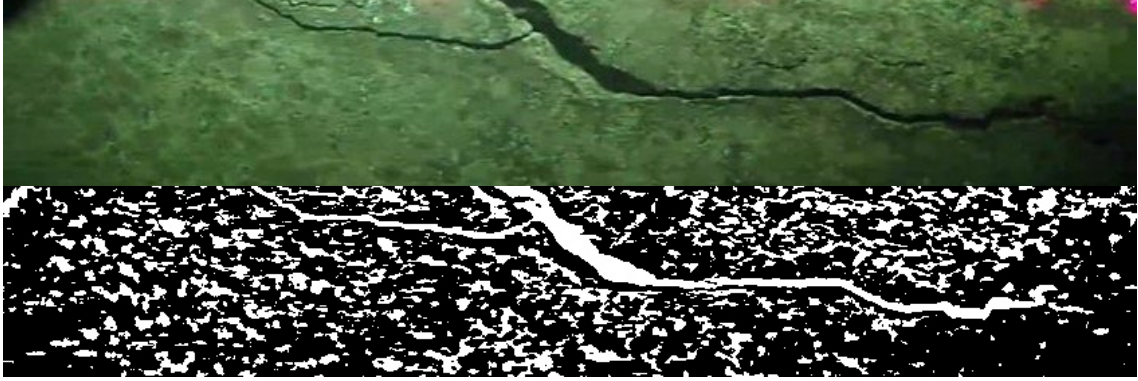


图 5 基于局部自适应阈值的二值化分割结果

2.4 数学形态学处理

2.4.1 形态学基本算子定义

为了修正二值化后的拓扑缺陷，本文引入数学形态学运算。形态学处理基于集合论，通过结构元素（Structure Element, B ）与图像集合（ A ）的相互作用来分析几何结构。基础运算定义如下：

- 腐蚀 (Erosion): $A \ominus B = \{z | (B)_z \subseteq A\}$
- 膨胀 (Dilation): $A \oplus B = \{z | (\hat{B})_z \cap A \neq \emptyset\}$

2.4.2 级联去噪与连通策略

本文设计了“先开后闭”的级联策略，代码实现见代码 4：

1. 开运算 (Opening, $A \circ B$)：先腐蚀后膨胀。本文使用 5×5 矩形核，主要用于滤除自适应阈值产生的孤立背景碎点。

2. 闭运算 (Closing, $A \bullet B$)：先膨胀后腐蚀。本文使用 9×3 非对称核，利用膨胀操作桥接裂缝断裂处，增强裂缝的连通性。

```
def main() -> None:
    input_path = resolve_input_path()
    image = cv2.imread(str(input_path))
    if image is None:
        raise SystemExit(f"Failed to read image: {input_path}")
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.medianBlur(gray, 3)
    binary = cv2.adaptiveThreshold(
        gray,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        41,
        5,
    )
    open_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    close_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (9, 3))
    opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, open_kernel, iterations=1)
    cleaned = remove_small_components(opened, min_area=120)
    closed = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, close_kernel, iterations=1)
```

代码 4 形态学开闭运算级联去噪与连通策略

经过级联形态学处理后，图像质量显著提升。如图 6 所示，裂缝区域的拓扑结构得到改善，孤立噪点被有效抑制，且断裂的裂缝段被连接成完整的曲线，为轮廓提取奠定了基础。



图 6 形态学级联运算后的去噪与连通性优化效果

2.5 轮廓提取与特征量化

为了从图像中获取裂缝的物理参数（如宽度、面积），需要将图像从“像素域”转换为“特征域”。考虑到水下原图的模糊性直接影响几何测量的精度，本节构建了“增强—提取—测量—统计”的一体化量化流程。

2.5.1 基于*Five A⁺* 的增强预处理

在进行轮廓提取之前，图像的边缘清晰度直接决定了量化结果的可靠性。传统的直方图均衡化等方法容易放大背景噪声，因此，本节首先引入 *Five A⁺* 深度学习模型作为高精度测量的基础前处理模块。

Five A⁺ Network 通过多尺度金字塔模块和频率域融合机制，能够有效去除水下图像的色偏与雾化效应，显著提升裂缝区域与大坝背景的对比度。我们将原始图像输入 *Five A⁺* 模块，生成的增强图像作为后续形态学分割与轮廓扫描的**高质量输入源**，确保了裂缝骨架的完整性，最大限度减少了断裂和误检。



图 7 *Five A⁺* Network 水下图像增强算法调用前后对比

2.5.2 拓扑轮廓扫描与几何筛选

基于增强后的图像，经阈值分割与形态学优化（见 2.2-2.4 节原理）得到二值图，随后采用 Suzuki 拓扑分析算法（`cv2.findContours`）扫描所有连通域的边界。

```
"""
Wldetect.py -> max_circle() 中的轮廓提取与列表推导式筛选部分
仅保留报告中引用的核心语句：Suzuki 轮廓跟踪 + 面积/细长比双约束。
"""

def code25_contour_extract_and_filter(thresh):
    contous, hierarchy = cv2.findContours(
        thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_TC89_L1
    )
    contous = [
        cnt
        for cnt in contous
        if cv2.contourArea(cnt) > 100
        and cv2.arcLength(cnt, True) / cv2.contourArea(cnt) > 0.1
    ]
    return contous, hierarchy
```

代码 5 基于几何特征的轮廓提取与筛选逻辑

为了进一步剔除背景纹理中残留的非裂缝噪点，本文基于裂缝的几何先验特征，设计了双重筛选准则（实现逻辑见代码 5）：

1. **面积约束：**设定 $Area(c_i) > 100$ ，滤除孤立的斑点噪声。
2. **细长比约束：**引入形状因子 $R = \text{ArcLength}/\text{Area}$ ，仅保留 $R > 0.1$ 的细长区域，精准锁定裂缝骨架。



图 8 裂缝检测结果

2.5.3 基于迭代内切圆的宽度测量

裂缝的**最大宽度**是评估大坝结构安全等级的核心指标。传统的基于骨架线距离变换（Distance Transform）的方法在边缘不平滑时易产生误差。本文在筛选出的裂缝轮廓内部，实现了一种高精度的**迭代最优内切圆算法**。

```
def iterated_optimal_incircle_radius_get(contous, pixelx, pixely, small_r, big_r,
precision):
    radius = small_r
    L = np.linspace(0, 2 * math.pi, 360) # 圆离散采样点
    circle_X = pixelx + radius * cos(L)
    circle_Y = pixely + radius * sin(L)
    for i in range(len(circle_Y)):
        if cv2.pointPolygonTest(contous, (circle_X[i], circle_Y[i]), False) < 0:
            return 0
    while big_r - small_r >= precision: # 二分法逼近最大内切圆半径
        half_r = (small_r + big_r) / 2
        circle_X = pixelx + half_r * cos(L)
        circle_Y = pixely + half_r * sin(L)
        if_out = False
        for i in range(len(circle_Y)):
            if cv2.pointPolygonTest(contous, (circle_X[i], circle_Y[i]), False) < 0:
                big_r = half_r
                if_out = True
        if not if_out:
            small_r = half_r
    return small_r
```

代码 6 迭代最优内切圆半径计算函数

该算法通过“网格化种子搜索 + 二分法逼近”的策略，寻找一个半径为 r_{max} 的全局最大内切圆（如图 9 所示），其直径即定义为裂缝的最大宽度：

$$W_{max} = 2 \times \max_{P \in \Omega} \{r \mid O(P, r) \subseteq \Omega_{crack}\}$$

其中 Ω_{crack} 为裂缝轮廓包围的区域。该方法能够鲁棒地定位出不规则裂缝中的“最危险截面”，有效克服了锯齿边缘对测量的干扰。



图 9 裂缝最大内切圆定位与宽度检测可视化

2.5.4 量化统计结果

系统最终将计算出的几何特征叠加在原图上进行可视化描述，并输出关键统计指标。这一过程完成了从图像像素到工程语义的跨越，输出如下：

裂缝分析结果	
裂缝数量:	2
裂缝占比:	3.59%
裂缝面积:	3097.5 像素
最大宽度:	17.2 像素
平均宽度:	11.6 像素

图 10 融合增强与传统算法的裂缝特征量化结果

```

def enhance_underwater_image(image_path: str,
                              model_path: str):
    enhancer = FiveAPlusEnhancer(model_path=model_path)
    out = enhancer.enhance_image(image_path)
    if out["success"] and out["enhanced_image"]:
        return out["enhanced_image"]
    return image_path

def detect_cracks(
    image_path: str,
    min_area: int = 200,
    threshold_value: int = 11,
    threshold_offset: int = 2,
    close_kernel_size: int = 5,
    close_iterations: int = 2,
):
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Failed to read image: {image_path}")
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    binary = cv2.adaptiveThreshold(
        blur,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        threshold_value,
        threshold_offset,
    )
    open_kernel = np.ones((3, 3), np.uint8)
    opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN,
                                open_kernel, iterations=1)
    close_kernel = np.ones((close_kernel_size,
                              close_kernel_size), np.uint8)
    closing = cv2.morphologyEx(opening,
                                cv2.MORPH_CLOSE, close_kernel,
                                iterations=close_iterations)
    continue

    cx = int(m["m10"] / m["m00"])
    cy = int(m["m01"] / m["m00"])
    dist = float(np.sqrt((cx - main_cx) ** 2 +
                          (cy - main_cy) ** 2))
    x, y, w, h = cv2.boundingRect(contour)
    overlap_x = max(0, min(main_bbox[2], x + w) -
                    max(main_bbox[0], x))
    overlap_y = max(0, min(main_bbox[3], y + h) -
                    max(main_bbox[1], y))
    if dist < max(img.shape[0], img.shape[1]) *
        0.3 or (overlap_x > 0 and overlap_y > 0):
        branch_contours.append(contour)
    selected = [main_contour] + branch_contours

    result_img = img.copy()
    cv2.drawContours(result_img, selected, -1, (0, 255,
                                                0), 2)
    result_path =
os.path.join(os.path.dirname(image_path), "crack_" +
os.path.basename(image_path))
    cv2.imwrite(result_path, result_img)
    total_area = float(img.shape[0] * img.shape[1])
    crack_area = float(sum(cv2.contourArea(c) for c in
selected))
    crack_percentage = (crack_area / total_area) *
100.0
    return {
        "result_path": result_path,
        "contours": selected,
        "crack_count": len(selected),
        "crack_area": round(crack_area, 2),
        "crack_percentage": round(crack_percentage, 2),
    }
    contours, _ = cv2.findContours(closing.copy(),
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    significant = [c for c in contours if
cv2.contourArea(c) > min_area]
    if not significant:
        result_path =
os.path.join(os.path.dirname(image_path), "crack_" +
os.path.basename(image_path))
        cv2.imwrite(result_path, img)
        return {"result_path": result_path, "contours":
[], "crack_count": 0, "crack_area": 0.0,
"crack_percentage": 0.0}
    sorted_contours = sorted(significant,
key=cv2.contourArea, reverse=True)
    main_contour = sorted_contours[0]
    branch_contours = []
    main_m = cv2.moments(main_contour)
    if main_m["m00"] != 0:
        main_cx = int(main_m["m10"] / main_m["m00"])
        main_cy = int(main_m["m01"] / main_m["m00"])
        x, y, w, h = cv2.boundingRect(main_contour)
        main_bbox = (x, y, x + w, y + h)

        for contour in sorted_contours[1:]:
            m = cv2.moments(contour)
            if m["m00"] == 0:

```

代码 7 轮廓提取与特征量化的核心代码

三、讨论：参数选择、局限与改进方向

3.1 参数设置的经验性与敏感性说明

本文方法以“平滑—分割—形态学—轮廓与统计”的传统流程为主线，整体具有较强的可解释性，但也不可避免地呈现出“参数驱动”的特点。尤其在水下图像中，亮度不均、背景纹理与噪声往往同时存在，使得同一组参数在不同区域的表现并不完全一致：有的区域裂缝能被较完整地连通，有的区域则可能出现断裂或纹理粘连。这种现象并不意味着流程不可用，而是说明在水下退化条件下，传统方法更依赖“输入可分离性”和“参数尺度与目标尺度的匹配”。

在预处理阶段，高斯平滑的核大小决定了降噪与保边之间的权衡。核偏小会导致噪声残留，从而在后续二值化中形成大量碎点；核偏大则会削弱裂缝边缘的局部对比，使细裂缝趋于模糊甚至被吞没。由于裂缝在图像中常表现为细长且宽度仅为若干像素的结构，平滑的尺度若超过裂缝自身尺度，往往会直接影响可检出性。因此，本文采用相对温和的平滑强度，并通过过程图观察“噪声颗粒是否被压制、裂缝主干是否仍连续”来确定合理范围。

在分割阶段，自适应阈值的窗口大小与偏移量（常数项）是最敏感的一组参数。窗口大小可以理解为局部统计的“参考尺度”：窗口过小会对纹理与噪声过度敏感，使二值图前景碎点显著增多；窗口过大则容易把光照渐变当成整体背景，导致裂缝的局部差异被平均化，从而出现断裂或漏检。偏移量则决定了前景判定的“松紧程度”：偏移量增大通常减少前景（更保守，漏检风险上升），偏移量减小则扩大前景（更激进，误检风险上升）。在水下图像中，由于背景纹理本身具有较强的局部对比，偏移量过小尤其容易把纹理误判为裂缝，这是本文后续必须引入形态学去噪的直接原因。

在形态学阶段，结构元素的形状与尺寸决定了“保留什么尺度的结构、连接什么尺度的断点”。开运算用于去除小噪点，但若结构元素过大，会把细裂缝当作“小结构”一并腐蚀掉；闭运算用于连通断裂，但若结构元素过大或迭代次数过多，会把相邻纹理区域误连成片，表现为粘连误检。由于裂缝具有明显的细长先验，采用长方形（非对称）结构元素可以在一定程度上强化“沿走向连接”的效果，但这同样依赖裂缝在图像中呈

现的主方向是否明显。因此本文将这些参数设计为可配置项，并以过程图（开运算、闭运算结果）作为“是否过腐蚀、是否过粘连”的直观判据来完成调参。

在特征量化阶段，面积阈值与细长比等筛选规则本质上是把“像素级前景”提升为“几何对象”的一道门槛。面积阈值过小会保留大量噪声连通域，使结果叠加图杂乱；面积阈值过大则可能将真实裂缝的细小段落误删，造成漏检。细长比约束能够抑制块状纹理，但在裂缝边界不清晰或闭运算发生粘连时，裂缝区域可能变得“偏胖”，从而被细长比规则误伤。至于宽度测量部分，本文采用迭代内切圆寻找最大内切圆直径来定义最大宽度，该方法对轮廓质量较敏感：若边界锯齿或存在小孔洞，会影响内切圆的稳定定位；同时其输出属于像素尺度，若需得到真实物理宽度，还需要标定与尺度换算。总体而言，本节所有参数均体现为“尺度匹配问题”：当参数尺度与裂缝尺度、噪声尺度的相对关系合适时，流程输出稳定；反之则会出现误检或漏检，这也是传统方法在复杂水下场景中的典型特性。

3.2 改进方向

针对上述局限，可以从三类方向改进。其一，在传统框架内提升鲁棒性：例如引入更稳定的对比度增强（如 CLAHE）、根据图像整体亮度/对比度自适应调整窗口与结构元素尺度，降低对人工调参的依赖。其二，增强裂缝细结构表达：在阈值分割前加入边缘信息（如梯度/边缘检测），或在得到二值裂缝后进行细化（骨架化），进一步提取裂缝长度、连通性等更贴近“细长目标”的指标。其三，在数据与算力允许时，可用学习型分割模型作为补充，但仍应保留传统流程用于可解释性检查与工程可控性（例如用传统结果做先验或做后处理约束）。

四、 结论

本文围绕《数字图像处理与机器视觉》课程的核心知识点，面向水下大坝表面裂缝这一典型目标，完成并实现了一套可解释的传统裂缝候选区域提取流程。实验从空间域平滑出发，逐步引入阈值分割与自适应阈值分割，并通过数学形态学的开闭运算对二值结果进行去噪与连通性修复，最后利用轮廓提取与几何筛选将像素级前景组织为可视化结果，同时输出裂缝数量、面积及占比等统计指标，实现了从“图像像素表示”到“可解释的目标区域与定量描述”的完整闭环。

实验结果表明，在水下退化与背景纹理较强的条件下，全局阈值方法容易受到光照不均影响而产生误分割；自适应阈值能够提升暗部裂缝的可检出性，但会引入较多碎点噪声，需要形态学后处理进行抑制与结构修复。整体流程能够输出较为连贯的裂缝候选轮廓，但对参数尺度较敏感，体现了传统方法在复杂场景中的典型特征。为提升输入可分离性并获得更可展示的最终效果，本文在最终综合结果生成阶段引入了水下增强模块作为工程前处理，并在保持后续传统流程一致的前提下完成识别与统计输出，验证了“增强前处理 + 传统流程”的组合在复杂条件下具有一定的实用性与可解释性。后续工作可从参数自适应、引入梯度/骨架等结构信息、以及建立物理尺度标定与多样本验证等方向继续改进，以进一步提升方法鲁棒性与量化可信度。

五、参考文献

- [1] GONZALEZ R C, WOODS R E. Digital Image Processing [M]. 4th ed. New York: Pearson, 2018.
- [2] PRATT W K. Digital Image Processing: PIKS Scientific Inside [M]. 4th ed. New Jersey: Wiley-Interscience, 2007.
- [3] OTSU N. A threshold selection method from gray-level histograms [J]. IEEE Transactions on Systems, Man, and Cybernetics, 1979, 9(1): 62-66.
- [4] CANNY J. A computational approach to edge detection [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986 (6): 679-698.
- [5] SERRA J. Image Analysis and Mathematical Morphology [M]. London: Academic Press, 1982.
- [6] SOILLE P. Morphological Image Analysis: Principles and Applications [M]. 2nd ed. Berlin: Springer, 2003.
- [7] BRADSKI G. The OpenCV Library [J]. Dr. Dobb's Journal of Software Tools, 2000.
- [8] LADKAT S. Introduction to Digital Image Processing, Computer Vision, and OpenCV [EB/OL]. (2020-04-19). <https://arsanatl.medium.com/introduction-to-digital-image-processing-computer-vision-and-opencv-92ce7383b24b>.
- [9] OWEN718. FiveA+Network: You Only Need 9K Parameters for Underwater Image Enhancement [EB/OL]. GitHub, 2023].<https://github.com/Owen718/FiveAPlus-Network>.

附录：代码仓库与运行说明

本项目的完整源代码及配套数据已上传至个人 Github 仓库，访问链接如下：

https://github.com/66Leslie/Course_ComputerVision-ImageProcessing

1. 实验环境配置

- 操作系统：Windows 10 / 11 (64-bit)
- 编程语言：Python 3.8+
- 核心依赖库（对应 [requirements.txt](#) / [environment.yml](#)）：
 - `opencv-python`（图像处理核心库，用于滤波/阈值/形态学/轮廓）
 - `numpy`（矩阵运算）
 - `flask`（若运行 [app.py](#) 的 Web 演示系统）
 - `torch / torchvision`（仅当启用 Five A+ 增强模块时需要）

2. 程序文件结构

— cracks.jpg	# 报告示例输入图
— 2_1_smoothing.py	# 2.1 平滑去噪
— 2_2_threshold.py	# 2.2 全局阈值/OTSU
— 2_3_adaptive_threshold.py	# 2.3 自适应阈值
— 2_4_morphology.py	# 2.4 形态学处理
— cracks_2_1_smooth.jpg	# 2.1 输出示例
— cracks_2_2_threshold.jpg	# 2.2 输出示例
— cracks_2_3_adaptive.jpg	# 2.3 输出示例
— cracks_2_4_opening.jpg	# 2.4 中间结果
— cracks_2_4_closing.jpg	# 2.4 中间结果
— code\	
— app.py	# Web 端集成演示入口（可选）
— Wldetect.py	# 轮廓/宽度量化相关实现（
— wldetect_wrapper.py	# 宽度检测封装
— five_aplus_wrapper.py	# Five A+ 增强封装（可选）
— Classification.py	# 分类训练脚本（工程内附）
— classification_wrapper.py	# 分类推理封装
— dam_crack_model.pth	# 分类模型权重
— requirements.txt	# pip 依赖
— environment.yml	# conda 环境依赖
— run.bat	# 启动脚本（可选）
— INSTALL.md	# 安装说明
— README.md	# 项目说明

3. 程序运行说明（生成报告 2.1–2.4 的结果图）

在 D:\project\Personal\COURSES\CV\report\ 目录下运行（脚本默认读取同目录的 cracks.jpg，也支持命令行传入图片路径）：

- `python 2_1_smoothing.py`
- `python 2_2_threshold.py`
- `python 2_3_adaptive_threshold.py`
- `python 2_4_morphology.py`